Presentation Overview

Title: Formal Verification meets AI, Using Lean as a Practical Bridge **Teacher:** Dániel Horpácsi **Presenter:** Aziz Najjar **Course:** Formal Semantics (Spring 2026) — Eötvös Loránd University **Estimated duration:** 15–20 minutes + Q&A

---

1. Objective

This presentation argues that the formal methods studied in our course, inference rules, operational and denotational semantics, compositionality, are directly relevant to an emerging and important problem: verifying that AI systems behave correctly. The proof assistant Lean 4 serves as the practical bridge between the two fields.

---

2. Structure

#	Section	Duration	Content
1	Hook	2 min	A real AI failure case + Dijkstra's "testing vs. proving" quote to motivate formal methods
2	Testing vs. Proving	3 min	Core distinction, connected to course material (denotational semantics covers *all* states, not just test cases)
3	From Course to Lean	2 min	Operational & denotational semantics recap → Lean automates the same inference-rule checking
4	Why Lean + AI	2 min	LeanDojo (NeurIPS 2023), AlphaProof (DeepMind 2024) — AI learning to prove in Lean
5	Three Code Examples	5–7 min	Pre-written Lean 4 code shown on slides and discussed line by line (see §3 below)
6	Big Picture	2 min	Timeline: LeanDojo → AlphaProof → DeepSeek-Prover-V2
7	Closing + Summary	1–2 min	"The next decade will ask: is your AI correct?"

---

3. The Three Lean 4 Code Examples

These are **not** live-coded. They appear as code snippets on slides and I will walk through them line by line, explaining each part. The code is real Lean 4 (verified with Mathlib) and kept deliberately minimal (3–8 lines each).

Example 1 — ReLU is Non-Negative

```
noncomputable def relu (x : ℝ) : ℝ := max 0 x

theorem relu_nonneg (x : ℝ) : relu x ≥ 0 := by
  unfold relu
  exact le_max_left 0 x
```

- **AI relevance:** ReLU is the standard neural network activation function.
- **Course connection:** Universal quantification over all inputs, like A⟦a⟧s being defined for all states.
- **Discussion point:** Two tactics, universal result — contrast with `assert relu(3) >= 0` (a test).

Example 2 — Vector Dimension Safety

```
def vecAdd (a b : Fin n → ℝ) : Fin n → ℝ :=
  fun i => a i + b i

#check vecAdd v3 v3   -- ✓ compiles
#check vecAdd v3 v2   -- ✗ TYPE ERROR
```

- **AI relevance:** Shape mismatch is the #1 runtime error in deep learning (PyTorch, TensorFlow).
- **Course connection:** Static semantics (Lecture 1, slide 20) — "constraints beyond context-free grammars."
- **Discussion point:** Errors caught at compile time vs. at 3 AM during a training run.

Example 3 — MSE Loss is Non-Negative

```
theorem mse_nonneg (ys ŷs : Fin n → ℝ) :
    0 ≤ ∑ i : Fin n, (ys i - ŷs i) ^ 2 := by
  apply Finset.sum_nonneg
  intro i _
  exact sq_nonneg _
```

- **AI relevance:** MSE is the most common regression loss function.
- **Course connection:** Compositional proof — mirrors how denotational semantics builds meaning from subexpressions (Lecture 3).
- **Discussion point:** `sum_nonneg` + `sq_nonneg` compose like semantic clauses.

---

4. References Used

Where in the talk	Reference	How it's used
Hook / motivation	AlphaProof (DeepMind, 2024)	AI solving IMO problems via Lean-based formal proof

Where in the talk	Reference	How it's used
Lean + AI section	LeanDojo (Yang et al., NeurIPS 2023)	Open toolkit: LLMs interacting with Lean to find proofs
Code examples	TorchLean (arXiv 2602.22631)	Inspiration for formally verified ML operations
Big picture / future	DeepSeek-Prover-V2 (arXiv, 2025)	State-of-the-art open-source theorem-proving model
Throughout	Course Lectures 1, 3, 4 (Horpácsi)	SOS rules, denotational semantics, compositionality, static semantics

5. Connections to Course Material

I explicitly tie the presentation back to our lectures at these points:

Course Topic	Where it appears	Bridge
Inference rules (Lec 3, 4)	Slides 4, 12	Lean tactics = applying inference rules
Compositionality (Lec 3)	Example 3 (MSE)	Proof composes from sub-proofs, like denotational semantics
Static semantics (Lec 1)	Example 2 (shapes)	Dependent types = stronger static semantics
States: Var → $\mathbb{Z}$ (Lec 3, 4)	Slide 12	Lean's local context ≈ state function
Derivation sequences (Lec 4)	Slide 12	A Lean tactic proof *is* a derivation sequence
Testing vs. proof (Lec 1)	Slide 3	Dijkstra's quote — the central motivation